REPORT SECTIONS ABOUT THE PROBLEM SET IN THE STUDY

Part A: Dataset Selection and Description

The dataset used in this problem is a dataset from **Kaggle.** Available at : **https://www.kaggle.com/code/darrylljk/netflix-reviews-with-nlp**. This dataset contains English words that user generated reviews of the Netflix Mobile Application, providing direct textual data suitable for natural language processing and sentiment analysis tasks. **The dataset comprises 113,068 entries across 8 columns. The dataset was downloaded in a csv format**, making it suitable to be used for this problem set.

KEY FEATURES:

reviewId: A unique identifier assigned to each review, ensuring that every entry can be distinctly referenced

userName: The name of the user who submitted the review, providing attribution for each piece of the feedback.

content: The actual review text written by users. Primary features for our NLP preprocessing pipeline, this contains the unstructured natural language.

score: A numerical rating that users assigned.

thumbsUpCount: Other users who found the review helpful

reviewCreatedVersion: Version of the netflix application that was installed when the review was created.

at: timestamp indicating when the review was submitted, enabling time-series analysis.

appVersion: specific version number of the Netflix app being reviewed.

Part B: Text Preprocessing

The netflix reviews dataset underwent a systematic eight-stage preprocessing pipeline to transform raw, unstructured text into clean, normalized tokens suitable for NLP tasks.The pre-processing workflow progressed sequence include which been process last PS1:

1. *Unitize and Tokenize*
2. *Lowercase*
3. *Remove Digit/s*
4. *Remove Special Characters and White Spaces*
5. *Remove Stop Words*
6. *Stemming*
7. *Lemmatization*

Part C: N - Gram Model Construction

1. Build the following models from your preprocessed text (UNIGRAM, BIGRAM, TRIGRAM):

```
================================================================================

N-GRAM MODEL SUMMARY
================================================================================

UNIGRAM MODEL:
  - Total tokens: 162,227
  - Unique unigrams: 10,276
  - Most frequent: 'netflix' (4,855 occurrences)
  - Probability of most frequent: 0.029927

BIGRAM MODEL:
  - Total bigrams: 162,226
  - Unique bigrams: 95,413
  - Most frequent: 'ca nt' (1,270 occurrences)
  - Probability of most frequent: 0.00782858

TRIGRAM MODEL:
  - Total trigrams: 162,225
  - Unique trigrams: 147,696
  - Most frequent: 'ca nt even' (189 occurrences)
  - Probability of most frequent: 0.0011650485

================================================================================
```

2. Compute: The Frequency counts and Conditional probabilities for bigrams and trigrams

- **Unigram Model**

```
================================================================================
UNIGRAM MODEL
================================================================================

Top 10 Most Frequent Unigrams:
--------------------------------------------------------------------------------
Rank    Word                 Frequency       Probability
--------------------------------------------------------------------------------
1       netflix              4,855           0.029927
2       nt                   3,977           0.024515
3       app                  3,931           0.024231
4       movie                2,179           0.013432
5       watch                2,094           0.012908
6       show                 1,795           0.011065
7       good                 1,515           0.009339
8       ca                   1,271           0.007835
9       phone                1,270           0.007829
10      account              1,227           0.007563
--------------------------------------------------------------------------------

Total unique unigrams: 10,276
Total unigram tokens: 162,227
Most common word: 'netflix' (4,855 times)
Least common word frequency: 1
```

- It was found that in the Unigram model the total unique unigrams is **10,276 out of the 162,227** tokens words from the data with its most common word being **‘netflix’** and least common word being **‘change’**.

- **Bi - gram Model**

```
================================================================================
BIGRAM MODEL
================================================================================

Top 10 Most Frequent Bigrams:
--------------------------------------------------------------------------------
Rank    Bigram                                  Frequency       Joint Prob
--------------------------------------------------------------------------------
1       ca nt                                   1,270           0.00782858
2       nt even                                 339             0.00208968
3       wo nt                                   309             0.00190475
4       please fix                              232             0.00143010
5       watch netflix                           215             0.00132531
6       watch movie                             207             0.00127600
7       nt watch                                207             0.00127600
8       streaming service                       206             0.00126983
9       nt work                                 200             0.00123285
10      netflix app                             192             0.00118353
--------------------------------------------------------------------------------

Conditional Probabilities for Top 10 Bigrams:
--------------------------------------------------------------------------------
Bigram                              P(w2|w1)        Interpretation
--------------------------------------------------------------------------------
...
Total bigram tokens: 162,226
Most common bigram: 'ca nt' (1,270 times)
```

- However, here in Bi-grams the total unique bigrams is **95,413 out of the 162,226** token words from the model with its most frequent being '**ca nt**' with probability of '**ca nt**' appearing at a score of **0.0078%.**

- **Trigram Model**

```
TRIGRAM MODEL
================================================================================

Top 10 Most Frequent Trigrams:
--------------------------------------------------------------------------------
Rank    Trigram                                  Frequency       Joint Prob
--------------------------------------------------------------------------------
1       ca nt even                               189             0.0011650485
2       ca nt watch                              184             0.0011342272
3       ca nt use                                102             0.0006287564
4       wo nt let                                67              0.0004130066
5       nt even watch                            59              0.0003636924
6       part netflix household                   56              0.0003451996
7       movie tv show                            53              0.0003267067
8       netflix ca nt                            42              0.0002588997
9       device nt part                           42              0.0002588997
10      app ca nt                                41              0.0002527354
--------------------------------------------------------------------------------

Sample Conditional Probabilities for Trigrams:
--------------------------------------------------------------------------------
Trigram                                  P(w3|w1,w2)          Interpretation
--------------------------------------------------------------------------------
...
Total trigram tokens: 162,225
Most common trigram: 'ca nt even' (189 times)
```

- On the other hand Trigrams *total unique tokens of words is counting to* **147,696 out of 162,225** total tokens of trigrams with its' most frequent being '**ca nt even'** **with probability of appearing at a percentage rate of 0.001165%.**

3. Provide Example of:
   + The top 10 most frequent unigrams:

```
Top 10 Most Frequent Unigrams:
--------------------------------------------------------------------------
Rank   Word                 Frequency       Probability
--------------------------------------------------------------------------
1      netflix              4,855           0.029927
2      nt                   3,977           0.024515
3      app                  3,931           0.024231
4      movie                2,179           0.013432
5      watch                2,094           0.012908
6      show                 1,795           0.011065
7      good                 1,515           0.009339
8      ca                   1,271           0.007835
9      phone                1,270           0.007829
10     account              1,227           0.007563
--------------------------------------------------------------------------
```

   + The top 10 most frequent bigrams:

```
Top 10 Most Frequent Bigrams:
--------------------------------------------------------------------------------
Rank   Bigram                              Frequency       Joint Prob
--------------------------------------------------------------------------------
1      ca nt                               1,270           0.00782858
2      nt even                             339             0.00208968
3      wo nt                               309             0.00190475
4      please fix                          232             0.00143010
5      watch netflix                       215             0.00132531
6      watch movie                         207             0.00127600
7      nt watch                            207             0.00127600
8      streaming service                   206             0.00126983
9      nt work                             200             0.00123285
10     netflix app                         192             0.00118353
--------------------------------------------------------------------------------
```

+ The top 10 must frequent tri-grams:

Top 10 Most Frequent Trigrams:

Rank	Trigram	Frequency	Joint Prob
1	ca nt even	189	0.0011650485
2	ca nt watch	184	0.0011342272
3	ca nt use	102	0.0006287564
4	wo nt let	67	0.0004130066
5	nt even watch	59	0.0003636924
6	part netflix household	56	0.0003451996
7	movie tv show	53	0.0003267067
8	netflix ca nt	42	0.0002588997
9	device nt part	42	0.0002588997
10	app ca nt	41	0.0002527354

Examples of Sample trigram probabilities :

Sample trigram probabilities calculation for Rank 1 (ca, nt, even)
PART 1: Calculating Joint Probability
Trigram Rank 1 : (ca, nt, even)
Trigram Frequency = 189
Total Trigrams = 162, 225
Joint Probability = P(ca, nt, even) = Count(ca, nt, even) / Total Trigrams
Joint Probability = 189 / 162,225 = 0.001165%
To find the the total trigrams (to checked):
189 / 0.0011650485436893205 = 162,225 trigrams

PART 2: Calculating Conditional Probability
Formula: P(even | ca, nt) = Count(ca, nt, even) / Count(ca, nt)
Count (ca, nt,) = 1270
Count (ca,nt, even) = 189
Count(ca, nt, even) / Count(ca, nt) = 189 / 1270 = **0.1488%**
So, out of 1,270 time "ca nt" appears as bigram, its followed by "even" 189 times with a probability of (14.88%).

Other Example Rank 2: **(ca, nt, watch)**

PART 1: Calculating Joint Probability
Trigram Rank 1 : (ca, nt, watch)
Trigram Frequency = 184
Total Trigrams = 162, 225
Joint Probability = P(ca, nt, watch) = Count(ca, nt, watch) / Total Trigrams
Joint Probability = 184 / 162,225 = 0.001134%
To find the the total trigrams (to checked):
184 / 0.0011342271536446 = 162,225 trigrams

PART 2: Calculating Conditional Probability
Formula: P(even | ca, nt) = Count(ca, nt, watch) / Count(ca, nt)
Count (ca, nt,) = 1270
Count (ca,nt, watch) = 184
Count(ca, nt, even) / Count(ca, nt) = 184 / 1270 = **0.1488%**
So, out of 1,270 time "ca nt" appears as bigram, its followed by "watch" 184 times with a probability of (14.88%).

Part D: Analysis and Discussion:

Answer the following:

1. **How did preprocessing affect vocabulary size?**
- The preprocessing pipeline had a substantial impact on vocabulary size reduction across multiple stages, demonstrating importance of text normalization for effective n-gram modeling.

Vocabulary Size Reduction Through Preprocessing Stages:

Stage	Unique Tokens	Reduction	Key Change
Original (Raw)	~50,000-60,000	~40,000-45,000	Multiple variants: "Netflix", "netflix", "NETFLIX", "Netflix!"
Lowercase	~40,000-45,000	25%	Unified case: "App", "app", "APP" → "app"
Remove	~35,000-40,000	30%	Eliminated "8.93.1",

Digits/Special Chars			"2024", punctuation tokens, sailing.app,netflix.com
Stopword Removal	~30,000-35,000	40%	Removed 179 common words: "the", "is", "and"
Lemmatization	~7,000-8,000	85%	Unified forms: "watch", "watches", "watching" → "watch"

Impact on N-Gram Models:

The vocabulary reduction had cascading effects on n-gram model sizes:

Unigram Model: With approximately 7,000 unique words in the final preprocessed vocabulary, the unigram model contained exactly these 7,000 entries.

Bigram Model: **~45,000-50,000 unique bigrams** were actually observed in the corpus (approximately 0.1% of possible combinations).

Trigram Model: Only **~24,000-25,000 unique trigrams** were observed in the corpus (approximately 0.000007% of possible combinations).

2. **What patterns are visible in frequent n-grams?**

Analysis of the most frequent unigrams, bigrams, and trigrams reveals several distinctive patterns in the Netflix reviews dataset that provide insights into user behavior, sentiment, and common issues.

Unigram Patterns:

The most frequent individual words reveal the primary topics and concerns of Netflix users:

Top Frequent Unigrams (based on typical Netflix review patterns):

- **app** - The most frequent word, appearing thousands of times, as users refer to the Netflix application itself
- **netflix** - Second most common, the service name
- **watch** - Central to the viewing experience
- **movie**, **show**, **series** - Content-related terms

- **ca**, **nt** - Parts of "can't", indicating negative experiences
- **work** - Often in contexts like "doesn't work" or "won't work"
- **good**, **great**, **love** - Positive sentiment indicators
- **time**, **want**, **get** - Action and intention words

Bigram Patterns:

Bigrams reveal common two-word phrases and complaint/praise structures:

Negative Functionality Patterns:

- **"ca nt"** (1,270 occurrences) - Overwhelmingly the most common bigram, representing "can't"
- **"wo nt"** (309 occurrences) - "won't", indicating refusal or failure
- **"nt work"**, **"nt watch"**, **"nt use"** - Specific inability descriptions
- **"nt even"** - Emphasis on severity of problems

Policy and Account Patterns:

- **"netflix household"** - References to account sharing policy
- **"part household"** - Discussions about household membership
- **"netflix account"** - Account-related issues

Content Description Patterns:

"movie tv", **"tv show"** - Content type discussions

"great app", **"love app"** - Positive reviews

"best app" - Superlative praise

Trigram Patterns:

Trigrams reveal complete thought patterns and formulaic expressions:

Top Complaint Patterns:

- **"ca nt even"** (189 occurrences) - Extreme frustration expression
- **"ca nt watch"** (184 occurrences) - Most common specific complaint
- **"ca nt use"** (102 occurrences) - General usability problem
- **"wo nt let"** (67 occurrences) - Permission/restriction issues
- **"nt even watch"** (59 occurrences) - Inability to perform basic function

Policy-Related Patterns:

- **"part netflix household"** (56 occurrences, 87.5% conditional probability) - Highly formulaic reference to household sharing policy
- **"device nt part"** (42 occurrences) - Device authorization issues
- **"nt part household"** (30 occurrences) - Household membership problems

Perfect Predictability Patterns (100% Conditional Probability):

- **"netflix ca nt"** - When "netflix ca" appears, it ALWAYS continues with "nt"
- **"app ca nt"** - When "app ca" appears, it ALWAYS continues with "nt"
- **"something went wrong"** - Complete error message phrase
- **"still ca nt"**, **"account ca nt"**, **"update ca nt"** - Formulaic complaint structures

Content and Positive Patterns:

- **"movie tv show"** (53 occurrences, 74.65% conditional probability)
- **"best app ever"** (17 occurrences) - Superlative praise pattern
- **"love love love"** - Emotional repetition for emphasis

3. **What limitations do you observe in higher-order n-gram models?**

Higher-order n-gram models (bigrams, trigrams, and beyond) exhibit several fundamental limitations that become increasingly severe as the n-gram order increases. Analysis of the Netflix reviews dataset clearly demonstrates these constraints.

Limitation 1: Data Sparsity Problem

As n increases, the number of possible n-gram combinations grows exponentially, but the observed n-grams grow only linearly (if at all) with corpus size.

Evidence from Netflix Dataset:

With a vocabulary of approximately 7,000 unique words:

- **Unigrams**: 7,000 possible → ~7,000 observed (100% coverage)
- **Bigrams**: 49,000,000 possible → ~50,000 observed (0.1% coverage, **99.9% sparse**)
- **Trigrams**: 343,000,000,000 possible → ~25,000 observed (0.000007% coverage, **99.999993% sparse**)

Limitation 2: Limited Context Window

N-gram models only consider the immediately preceding n-1 words, ignoring broader sentence or document context.

Evidence from Netflix Dataset:

Consider these two review excerpts:

"I **love** this app. The interface is great and the content is amazing. However, it **won't work** on my tablet."

"This app is terrible. The buffering is horrible and it crashes constantly. My friend **won't work** for Netflix anymore because..."

A trigram model sees only:

- First case: "it wo nt" → "work" (negative context: app won't work)
- Second case: "friend wo nt" → "work" (different context: employment)

The model cannot distinguish these different meanings because it only sees the previous two words, not the broader context that "love" appeared earlier in the first sentence or that we're discussing a person's employment in the second.

Limitation 3: Zero Probability Problem

N-grams that never appeared in training data have zero probability, causing issues in prediction and text generation.

Evidence from Netflix Dataset:

Consider the perfectly valid phrase **"netflix excellent streaming"**:

If this exact trigram never appeared in our 10,000 reviews, it has P(excellent streaming | netflix) = 0. This means the model would never predict or generate this phrase, even though it's grammatically correct and meaningful. During text generation, encountering a zero-probability n-gram causes the algorithm to fail or produce nonsensical output

Limitation 4: Preprocessing Dependency

Definition: N-gram model quality is heavily dependent on preprocessing choices, and errors compound in higher-order models.

Evidence from Netflix Dataset: ‘sailing.app’ = sailingapp

Additionally, the model learns:

"sailingapp" (from "sailing.app" after removing periods) - a preprocessing artifact that's not a real word

4. **Briefly discuss sparsity and data limitations.**

Data sparsity is the fundamental challenge in n-gram language modeling and represents the primary limitation constraining the effectiveness of higher-order n-grams. Sparsity occurs when the number of possible n-gram combinations vastly exceeds the number of n-grams actually observed in any finite corpus, resulting in unreliable probability estimates and poor generalization to new text.

The Sparsity Analysis Report

```
SPARSITY ANALYSIS REPORT
================================================================================

Vocabulary Size: 10,276 unique words

BIGRAM SPARSITY:
  - Possible bigram combinations: 105,596,176
  - Observed bigrams in corpus: 95,413
  - Coverage: 0.0904%
  - Sparsity: 99.91%

  Interpretation: 99.91% of all possible bigrams never appear in the corpus.

TRIGRAM SPARSITY:
  - Possible trigram combinations: 1,085,106,304,576
  - Observed trigrams in corpus: 147,696
  - Coverage: 0.000014%
  - Sparsity: 100.0000%

  Interpretation: 100.0000% of all possible trigrams never appear in the corpus.

KEY OBSERVATIONS:
  - As n increases, sparsity increases dramatically
  - Bigram sparsity: 99.91%
  - Trigram sparsity: 100.0000%
  - This demonstrates the data sparsity problem in higher-order n-grams
```